

Цель работы: сформировать умения внедрять сценарии в HTML-документ, обрабатывать события с использованием DHTML.

Выполнение заданий:

Вариант 11

Задание 1

1. Создайте класс с именем «Contacts» для будущего приложения «Контакты», и создайте класс с именем «User» для каждого отдельного контакта.

2. В классе «User» реализуйте сл. свойства и методы:

1. Свойство data – для хранения данных о контакте в виде простого объекта со сл. полями: id, name, email, address, phone.

2. Метод edit(obj) – для редактирования данных контакта. В качестве параметра метод должен принимать объект с обновленными данными и обновлять свойство data.

3. Метод get() – для получения данных о контакте. Метод должен возвращать объект с данными из св-ва data.

4. При создании объекта на основе этого класса важно передать в конструктор данные о контакте в виде объекта для дальнейшего сохранения в св-во data.

3. В классе «Contacts» реализуйте сл. св-ва и методы:

1. Свойство data – для хранения всех добавленных контактов в массиве. Каждый эл-т массива должен представлять собой объект созданный на основе класса «User».

2. Метод add() – для создания контакта (на основе класса «User») и добавления его в массив data.

3. Метод edit(id, obj) – для редактирования информации конкретного контакта из св-ва data, используя соответствующий метод из «User». В качестве параметра нужно передать идентификатор контакта для последующего поиска и объект с новыми данными для редактирования.

4. Метод remove(id) – для удаления контакта из общего массива данных по идентификатору. В качестве параметра нужно передать идентификатор контакта.

5. Метод get() – для получения всех контактов. Метод должен возвращать объект с данными из св-ва data.

4. Создайте еще один класс «ContactsApp» для создания интерфейса и показа его в браузере. Класс должен наследоваться от «Contacts».

5. В классе «ContactsApp» реализуйте сл. св-ва и методы:

1. При создании объекта на основе этого класса в DOM должен добавляться главный контейнер приложения, например, «<div class="contacts"></div>». Доступ к контейнеру должен быть через свойство «app» (должен хранить созданный элемент).

2. Также, в конструкторе или через любой метод в классе полностью создайте интерфейс вашего приложения внутри главного контейнера. Предусмотрите форму с полями и кнопками для добавления и редактирования контактов. Дизайн может быть любым, но адаптивным к мобильным устройствам.

3. Методы onAdd(), onEdit() и onRemove() – должны срабатывать по клику по соотв. кнопкам в интерфейсе для добавления/редактирования/удаления контакта. Важно использование методов от «Contacts» при соотв. действиях.

4. Метод get() – для получения и обновления списка контактов в соотв. контейнере вашего приложения. Важно сохранить возможности родительского метода.

Вы можете добавлять любые другие свойства и методы во все классы.

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Контакты</title>
  </head>
  <body>
    <h1>Контакты</h1>
    <button id="addBtn" onclick="ca.onAdd()">Добавить</button>
    <button id="editBtn" onclick="ca.onEdit()">Изменить</button>
    <button id="delBtn" onclick="ca.onRemove()">Удалить</button>
    <p style="font-weight: bold">_____</p>
    <div id="contacts"></div>
  </body>
  <script src="script.js"></script>
</html>
```

script.js

```
class User {
  constructor(data) {
    this.data.name = data.name;
    this.data.email = data.email;
    this.data.address = data.address;
    this.data.phone = data.phone;
  }
  data = {
    name: "John Smith",
    email: "john@example.com",
    address: "John Smith's Street Address",
    phone: "123-456-789",
  };
  edit(obj) {
    obj.data = obj;
  }
  get() {
    return data;
  }
}

class Contacts {
  data = [];
  constructor() {}
  add() {
    let data = {
      name: prompt("Имя"),
      email: prompt("Почта"),
      address: prompt("Адрес"),
      phone: prompt("Телефон"),
    };
    this.data.push(new User(data));
  }
}
```

```

    }
    edit(id, obj) {
        this.data[id - 1].edit(obj);
    }
    remove(id) {
        this.data.splice(this.data.indexOf(id - 1), 1);
    }
    get() {
        return this.data;
    }
}

class ContactsApp {
    app = document.getElementById("contacts");
    contacts;
    constructor() {
        this.contacts = new Contacts();
    }
    onAdd() {
        this.contacts.add();
        let last = this.contacts.get().length - 1;
        console.log(last);
        let dataObj = this.contacts.get()[last].data;

        const newDiv = document.createElement("div");
        newDiv.setAttribute("id", last + 1);
        newDiv.appendChild(document.createTextNode("№ " + (last + 1) + "| Name: " +
dataObj.name + "; Email: " + dataObj.email + "; Address" + dataObj.address + "; Phone: "
+ dataObj.phone));
        document.getElementById("contacts").appendChild(newDiv);
    }
    onEdit() {
        let id = prompt("Введите порядковый номер в списке контактов");
        let dataObj = {
            name: prompt("Имя"),
            email: prompt("Почта"),
            address: prompt("Адрес"),
            phone: prompt("Телефон"),
        };
        this.contacts.edit(id, dataObj);
        document.getElementById(id).innerHTML = "№ " + id + "| Name: " + dataObj.name +
"; Email: " + dataObj.email + "; Address" + dataObj.address + "; Phone" + dataObj.phone;
    }
    onRemove() {
        let id = prompt("Введите порядковый номер в списке контактов");
        this.contacts.remove(id);
        document.getElementById(id).remove();
    }
    get() {}
}

var ca = new ContactsApp();

```